

```
1. import java.io.*;
```

```
import java.util.*;
```

```
class Result {
```

```
    public static long stringSimilarity(String s) {
```

```
        int n = s.length();
```

```
        long sum = 0;
```

```
        for (int i = 0; i < n; i++) {
```

```
            int j = 0;
```

```
            while (i + j < n && s.charAt(j) == s.charAt(i + j)) {
```

```
                j++;
```

```
            }
```

```
            sum += j;
```

```
        }
```

```
        return sum;
```

```
    }
```

```
}
```

```
public class Solution {
```

```
    public static void main(String[] args) throws IOException {
```

```
        BufferedReader br =
```

```
            new BufferedReader(new InputStreamReader(System.in));
```

```

int t = Integer.parseInt(br.readLine());

while (t-- > 0) {

    String s = br.readLine().trim();

    System.out.println(Result.stringSimilarity(s));
}

br.close();
}

```

Input (stdin)	
1	2
2	ababaa
3	aa
Your Output (stdout)	
1	11
2	3
Expected Output	
1	11
2	3

```

2. class Solution {
    public boolean repeatedSubstringPattern(String s) {

        int n = s.length();

        for (int len = 1; len <= n / 2; len++) {

            if (n % len != 0) {

```

```
        continue;
    }

    String sub = s.substring(0, len);

    boolean valid = true;

    for (int i = len; i < n; i += len) {

        if (!s.substring(i, i + len).equals(sub)) {
            valid = false;
            break;
        }
    }

    if (valid) {
        return true;
    }

    return false;
}
}
```

Input

```
s =  
"abab"
```

Output

```
true
```

Expected

```
true
```

```
3. import java.io.*;  
import java.math.*;  
import java.security.*;  
import java.text.*;  
import java.util.*;  
import java.util.concurrent.*;  
import java.util.function.*;  
import java.util.regex.*;  
import java.util.stream.*;  
import static java.util.stream.Collectors.joining;  
import static java.util.stream.Collectors.toList;  
  
class Result {  
  
    public static String twoStrings(String s1, String s2) {  
  
        HashSet<Character> set = new HashSet<>();  
  
        // Store characters of s1  
        for (char c : s1.toCharArray()) {  
            set.add(c);  
        }  
    }  
}
```

```

// Check characters of s2
for (char c : s2.toCharArray()) {
    if (set.contains(c)) {
        return "YES";
    }
}

return "NO";
}
}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));

        BufferedWriter bufferedWriter = new BufferedWriter(new
        FileWriter(System.getenv("OUTPUT_PATH")));

        int q = Integer.parseInt(bufferedReader.readLine().trim());

        IntStream.range(0, q).forEach(qltr -> {
            try {
                String s1 = bufferedReader.readLine();

                String s2 = bufferedReader.readLine();

                String result = Result.twoStrings(s1, s2);

                bufferedWriter.write(result);
                bufferedWriter.newLine();
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        });
    }
}

```

```

        }
    });

    bufferedReader.close();
    bufferedWriter.close();
}
}

```

```

2  hello
3  world
4  hi
5  world

our Output (stdout)
1  YES
2  NO

xpected Output
1  YES
2  NO

```

```

4. class Solution {
    public boolean rotateString(String s, String goal) {

        if (s.length() != goal.length()) {
            return false;
        }

        String combined = s + s;

        return combined.contains(goal);
    }
}

```

Input

```
s =  
"m"
```

```
goal =  
"f"
```

Output

```
false
```

Expected

```
false
```

```
5.import java.io.*;  
import java.math.*;  
import java.security.*;  
import java.text.*;  
import java.util.*;  
import java.util.concurrent.*;  
import java.util.function.*;  
import java.util.regex.*;  
import java.util.stream.*;  
import static java.util.stream.Collectors.joining;  
import static java.util.stream.Collectors.toList;
```

```
class Result {
```

```
    public static int marsExploration(String s) {
```

```
        int count = 0;
```

```
        String pattern = "SOS";
```

```

        for (int i = 0; i < s.length(); i++) {

            if (s.charAt(i) != pattern.charAt(i % 3)) {

                count++;

            }

        }

        return count;

    }

}

public class Solution {

    public static void main(String[] args) throws IOException {

        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));

        BufferedWriter bufferedWriter = new BufferedWriter(new
        FileWriter(System.getenv("OUTPUT_PATH")));

        String s = bufferedReader.readLine();

        int result = Result.marsExploration(s);

        bufferedWriter.write(String.valueOf(result));

        bufferedWriter.newLine();

        bufferedReader.close();

        bufferedWriter.close();

    }

}

```



6. import java.util.*;

```
class Solution {  
    public List<Integer> findAnagrams(String s, String p) {  
        List<Integer> result = new ArrayList<>();  
  
        if (s.length() < p.length()) {  
            return result;  
        }  
  
        int[] pCount = new int[26];  
        int[] window = new int[26];  
  
        // Count characters in p  
        for (char c : p.toCharArray()) {  
            pCount[c - 'a']++;  
        }  
  
        int k = p.length();  
  
        // First window  
        for (int i = 0; i < k; i++) {
```

```

        window[s.charAt(i) - 'a']++;
    }

    // Check first window
    if (Arrays.equals(pCount, window)) {
        result.add(0);
    }

    // Slide the window
    for (int i = k; i < s.length(); i++) {

        // Add new character
        window[s.charAt(i) - 'a']++;

        // Remove old character
        window[s.charAt(i - k) - 'a']--;

        // Check if anagram
        if (Arrays.equals(pCount, window)) {
            result.add(i - k + 1);
        }
    }

    return result;
}
}

```

Input

s =
"cbaebabacd"

p =
"abc"

Output

[0,6]

Expected

[0,6]

```
7. import java.util.*;
```

```
public class Solution {
```

```
    public static int palindromeIndex(String s) {
```

```
        int left = 0;
```

```
        int right = s.length() - 1;
```

```
        while (left < right) {
```

```
            if (s.charAt(left) != s.charAt(right)) {
```

```
                // Remove left character
```

```
                if (isPalindrome(s, left + 1, right)) {
```

```
                    return left;
```

```
                }
```

```
                // Remove right character
```

```
                if (isPalindrome(s, left, right - 1)) {
```

```
                    return right;
```

```
                }
```

```

        return -1;
    }

    left++;
    right--;
}

return -1;
}

public static boolean isPalindrome(String s, int left, int right) {

    while (left < right) {

        if (s.charAt(left) != s.charAt(right)) {
            return false;
        }

        left++;
        right--;
    }

    return true;
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    int q = sc.nextInt();

```

```

while (q > 0) {

    String s = sc.next();

    System.out.println(palindromeIndex(s));

    q--;
}

sc.close();
}

```

Input (stdin)	
1	3
2	aaab
3	baa
4	aaa

Expected Output	
1	3
2	0
3	-1

```

8. class Solution {
    public int strStr(String haystack, String needle) {

        int n = haystack.length();
        int m = needle.length();

        for (int i = 0; i <= n - m; i++) {

            int j = 0;

```

```

        while (j < m && haystack.charAt(i + j) == needle.charAt(j)) {
            j++;
        }

        if (j == m) {
            return i;
        }
    }

    return -1;
}

```

Input

```

haystack =
"sadbutsad"

```

```

needle =
"sad"

```

Output

```
0
```

Expected

```
0
```

9. class Solution {

```

    public String longestPalindrome(String s) {

```

```

        if (s.length() < 2) {
            return s;
        }

```

```

int start = 0;

int end = 0;

for (int i = 0; i < s.length(); i++) {

    // Odd length palindrome
    int len1 = expand(s, i, i);

    // Even length palindrome
    int len2 = expand(s, i, i + 1);

    int len = Math.max(len1, len2);

    if (len > end - start + 1) {

        start = i - (len - 1) / 2;
        end = i + len / 2;
    }
}

return s.substring(start, end + 1);
}

public int expand(String s, int left, int right) {

    while (left >= 0 && right < s.length()
        && s.charAt(left) == s.charAt(right)) {

        left--;
        right++;
    }
}

```

```

    }

    return right - left - 1;
}
}

```

Input

```

s =
"babad"

```

Output

```

"bab"

```

Expected

```

"bab"

```

```

10. import java.io.*;
import java.util.*;

```

```

class Result {

```

```

    public static List<Integer> circularPalindromes(String s) {

```

```

        int n = s.length();

```

```

        // Double the string to handle rotations

```

```

        String str = s + s;

```

```

        // Build transformed string for Manacher's algorithm

```

```

        StringBuilder t = new StringBuilder();

```

```

        t.append('#');

```

```

for (int i = 0; i < str.length(); i++) {
    t.append(str.charAt(i));
    t.append('#');
}

int m = t.length();

int[] p = new int[m];

int center = 0;
int right = 0;

// Manacher's algorithm
for (int i = 0; i < m; i++) {

    int mirror = 2 * center - i;

    if (i < right) {
        p[i] = Math.min(right - i, p[mirror]);
    }

    while (i - p[i] - 1 >= 0 &&
        i + p[i] + 1 < m &&
        t.charAt(i - p[i] - 1)
            == t.charAt(i + p[i] + 1)) {

        p[i]++;
    }

    if (i + p[i] > right) {
        center = i;
    }
}

```

```
        right = i + p[i];  
    }  
}
```

```
List<Integer> answer = new ArrayList<>();
```

```
// Check every rotation
```

```
for (int start = 0; start < n; start++) {
```

```
    int max = 1;
```

```
    /*
```

```
     * Odd length palindromes.
```

```
     *
```

```
     * Character i is at position 2*i+1
```

```
     * in transformed string.
```

```
    */
```

```
    for (int i = start; i < start + n; i++) {
```

```
        int radius = p[2 * i + 1] / 2;
```

```
        // Cannot go outside this rotation
```

```
        int left = i - start;
```

```
        int rightSide = start + n - 1 - i;
```

```
        radius = Math.min(radius,
```

```
            Math.min(left, rightSide));
```

```
        int length = 2 * radius + 1;
```

```
        max = Math.max(max, length);
```

```

    }

    /*
    * Even length palindromes.
    *
    * Center is between i-1 and i.
    */
    for (int i = start + 1; i < start + n; i++) {

        int radius = p[2 * i] / 2;

        int left = i - start;
        int rightSide = start + n - i;

        radius = Math.min(radius,
            Math.min(left, rightSide));

        int length = 2 * radius;

        max = Math.max(max, length);
    }

    answer.add(max);
}

return answer;
}
}

public class Solution {

```

```
public static void main(String[] args) throws Exception {

    BufferedReader br =
        new BufferedReader(
            new InputStreamReader(System.in));

    int n = Integer.parseInt(br.readLine().trim());

    String s = br.readLine().trim();

    List<Integer> result =
        Result.circularPalindromes(s);

    StringBuilder output = new StringBuilder();

    for (int x : result) {
        output.append(x).append('\n');
    }

    System.out.print(output);
}
```

Expected Output

1	12
2	12
3	10
4	8
5	8
6	9
7	11
8	13
9	11
10	9
11	8
12	8